

Extending VS Math Models with VS Commands and the VS API

An Example Model Extension	2
Existing Model Documentation	3
Following a Path using a Preview Point	3
Steer Controller in CarSim and TruckSim	4
BikeSim Lean and Steer	4
Access the Examples	5
Adding the Controller with VS Commands	7
VS Commands for the Example	7
Running the Extended Model	9
Extending a VS Model with a Custom Wrapper: Overview	10
Extending a VS Model with MATLAB	11
Review: Running a VS Math Model using MATLAB	11
Running a Simulation Loop in MATLAB	12
Applying VS Commands from MATLAB	14
Using Import and Export Arrays with MATLAB	16
Extending a VS Model with C/C++	21
Review: Running a C Wrapper EXE Program	21
Applying VS Commands from a C Wrapper Program	22
Using Callback Functions from a C Wrapper Program	23
Summary	29
VS Commands, No External Software	29
MATLAB with VS Commands	30
MATLAB with I/O Arrays	30
C with VS Commands	31
C with VS API Functions	31

The vehicle VS Solver libraries for CarSim, TruckSim, and BikeSim are used to build and run VS Math Models to predict the behavior of conventional vehicles in response to driver/rider controls.

There are at least four approaches for extending a VS Math Model beyond the built-in features:

1. The VS Library can be run from external simulation software such as Simulink that combines the VS Math Model with modeling in the external workspace.
2. Equations can be added at runtime using VS Commands and embedded Python.
3. Variables can be exchanged with an external custom program written in any programming language that can load a Windows dynamically linked library (DLL) file and access its functions. This is the same technique as used with Simulink, applied with user-defined custom software.

4. Variables in the VS Math Model can be accessed directly by a custom program written in C/C++. Calculations in the custom program can be tightly integrated with the calculations built into the VS Math Model.

This memo demonstrates three of the approaches using a simple path-following controller. (The Simulink approach is not covered here.)

VS Commands are described in the *VS Commands Reference Manual* and require no additional software beyond a VehicleSim product from Mechanical Simulation such as CarSim. The other methods are described in the reference manual *VehicleSim API: Running and Extending VS Solver Programs*. The example programs and datasets from this memo are included in CarSim, TruckSim, and BikeSim.

An Example Model Extension

A path preview control is used as an example model extension. The controller is based on a “preview point” in front of the vehicle and the relationship of that preview point to a target path relative to a road reference line. If the point is not on target, an angle is calculated that is proportional to the lateral distance between the point and the target lateral position.

Figure 1 shows the point represented with an arrow in front of a CarSim vehicle. The intended location is the center of the right-hand lane. The arrow is a little to the left of the target line, and therefore the vehicle steering wheel should be turned to the right. In the case of BikeSim, where control is made by leaning the bike, the bike should be leaned to the right.

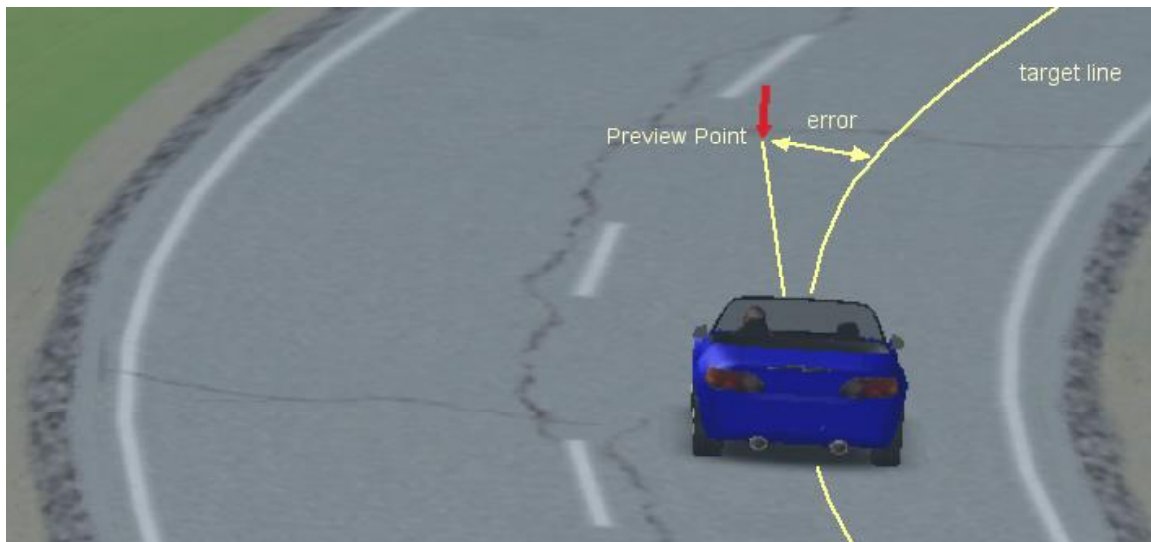


Figure 1. Vehicle with a preview point for a simple steering controller.

Note	The example controller presented in this memo is provided solely to illustrate the use of VS Commands and the VS API. It is not intended to replace the controllers that are already built into the products.
-------------	---

Existing Model Documentation

The full list of available Import variables for a VS Math Model is generated by using the **View** button in the lower-right corner of the **Run Control** screen. Two versions are available: one for viewing with a text editor, and one for viewing as a spreadsheet.

If you browse the documentation file for any of the vehicle models in CarSim or TruckSim, you will see that they all include steering wheel angle as a variable that can be imported. In BikeSim, you can import steering torque and target lean angle.

The default settings for all the Import variables are that they be ignored. Although the native equations in the VS Math Model include the effect of the imported variables, they do not set values for the import variables. (After all, the whole purpose of importing a variable is to replace or modify the value already available in the native equations of the model.)

The output variables available in a VS Math Model can also be viewed using the **View** button on the **Run Control** screen. Reading through the list of available variables shows that there are many X and Y coordinates for points such as the origin of the sprung mass coordinate system, the vehicle center of gravity (CG), etc. A preview point for a new steering controller can be based on any of the existing X and Y coordinate pairs. In this example, the X and Y coordinates of the vehicle CG will be used for CarSim and TruckSim; for BikeSim, the coordinates of the rider reference point are used.

Following a Path using a Preview Point

The example controllers will attempt to follow a path based on difference between a target lateral distance from a path (specified with a new parameter `LAT_TRACK`), and the lateral distance from a preview point to the path, where the point is a specified distance in front a point of interest on the vehicle. For this example, the coordinates of the preview point are:

$$X_{\text{preview}} = X_o + L_FORWARD * \cos(Yaw) \quad (1)$$

$$Y_{\text{preview}} = Y_o + L_FORWARD * \sin(Yaw) \quad (2)$$

where

X_{preview} and Y_{preview} are X and Y coordinates of a point in front of the vehicle (new output variables),

X_o and Y_o are the names of existing output variables (these are also state variables in the math model) that provide the X and Y coordinates of the origin of the coordinate system of the sprung mass, as listed in a documentation file obtained using the **View** button on the **Run Control** screen,

Yaw is the output variable for the yaw angle for the vehicle (also a state variable), as listed in the same documentation file, and

$L_FORWARD$ is a new parameter for the distance the preview point lies in front of the reference point in the sprung mass.

Steer Controller in CarSim and TruckSim

Directional control in CarSim and TruckSim is provided by either steering wheel torque or steering wheel angle, depending on the selected mode. The example controllers use steering wheel angle.

The imported steering wheel angle can be activated by providing a line of text in any data field that provides input to the model:

```
IMPORT IMP_STEER_SW VS_REPLACE (3)
```

The example extensions will calculate the imported steering wheel angle, based on the position of the target point, which in turn depends on the current vehicle position and heading angle.

The desired steer angle during the run will be set using the import:

```
IMP_STEER_CTRL = GAIN_PATH_CTRL*  
(LAT_TRACK -ROAD_L_ID(Xpreview, Ypreview, CURRENT_ROAD_ID, 1)) (4)
```

where

LAT_TRACK is a new parameter for the target lateral position relative to the road path,

ROAD_L_ID is a function available in VS Math Models that gives the lateral distance of a point defined by X and Y coordinates relative to the road reference line (typically the centerline) for a specified Road ID,

CURRENT_ROAD_ID is a parameter for the ID of the current road, and

GAIN_PATH_CTRL is a new parameter used to scale the control in proportion to tracking error.

The controller calculation is potentially complicated at the start of the run when not all the output variables have been calculated. For this simple example, the steering wheel angle will be defined as zero until the simulation time T is greater than the start time parameter T_START . Therefore, equation 4 is replaced with a more complicated expression:

```
IMP_STEER_SW = IF(T > TSTART,  
    GAIN_PATH_CTRL*  
    (LAT_TRACK -ROAD_L_ID(Xpreview, Ypreview, CURRENT_ROAD_ID, 1)),  
    0) (5)
```

This uses the IF special form to set the steer to 0 during initialization, before $T > T_START$.

BikeSim Lean and Steer

BikeSim has two levels of control for following a path:

1. Lean angle provides the control of the bike direction. An internal rider model (enabled with the keyword OPT_RIDER_MODEL) determines a lean target GAMMA_TARG needed to follow a specified path.
2. A steer controller (enabled with the keyword OPT_STEER_CONTROL) determines a steer torque from an internal model to try to obtain the target lean angle GAMMA_TARG.

Almost the same path follower logic is used in BikeSim, except that the controller angle is used as target lean. The Import variable is named IMP_GAMMA_TARG. The parameter

GAIN_PATH_CTRL for BikeSim has the same units of deg/m. There is a difference in sign convention compared to CarSim/TruckSim; steering to the left is positive but leaning to the left is negative. Hence, the equation for lean has a minus sign:

$$\text{IMP_GAMMA_TARG} = \text{IF}(\text{T} > \text{TSTART}, \\ -\text{GAIN_PATH_CTRL} * \\ (\text{LAT_TRACK} - \text{ROAD_L_ID}(\text{Xpreview}, \text{Ypreview}, \text{CURRENT_ROAD_ID}, 1)), \\ 0) \quad (6)$$

Most of the following descriptions involve CarSim. Similar examples for BikeSim are included in the databases, with the distinctions being:

1. The BikeSim controller provides target lean angle, rather than steering angle,
2. The BikeSim controller uses equation 6, rather than equation 5.
3. The internal steer controller is enabled to apply steering torque to try to achieve the lean angle calculated in equation 6.

Access the Examples

When working from a Windows installation with a VS Browser, use the File menu to select the option **File > Open Database Builder**. This brings up the **Database Builder** window (Figure 2).

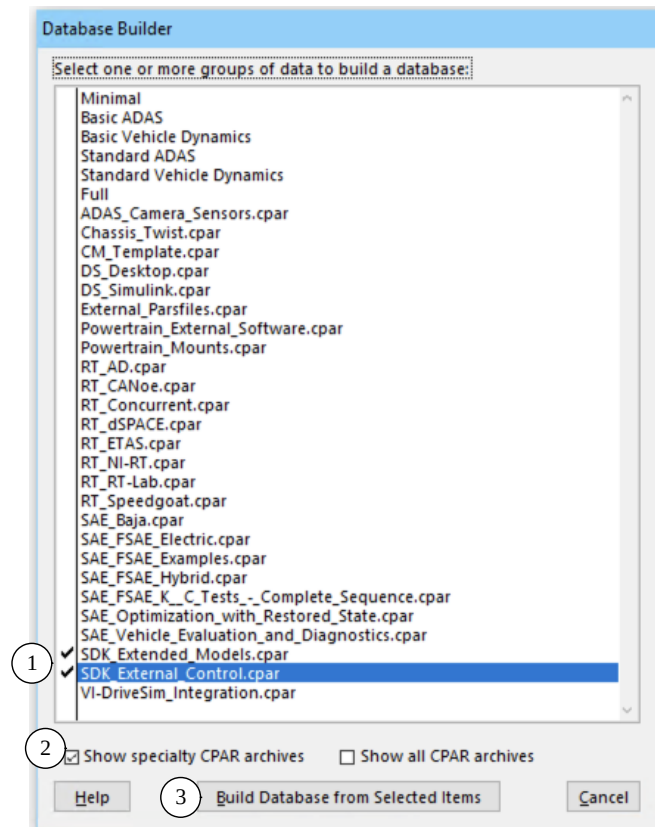


Figure 2. The Database Builder window with the two SDK CPAR archives selected.

Check the button **Show specialty CPAR archives** (2) to view the CPAR archives that are not part of the default database options. Select the two SDK archives (1), then click the **Build Database** button (3) to proceed. The next steps are the same as used to create any database from the VS Browser and should be familiar.

These two CPAR archives will provide runs in four categories:

1. SDK Extended Models,
2. SDK External Control,
3. Extended Models, and
4. Quick Start Guide.

Next, you want to view the folders that were imported. Choose the item from the VS Browser **View** menu (1) (Figure 3) **View > Open this Library Folder in Windows** (2). This will take you to the Windows viewer showing the folder associated with the library currently in view (Figure 4a).

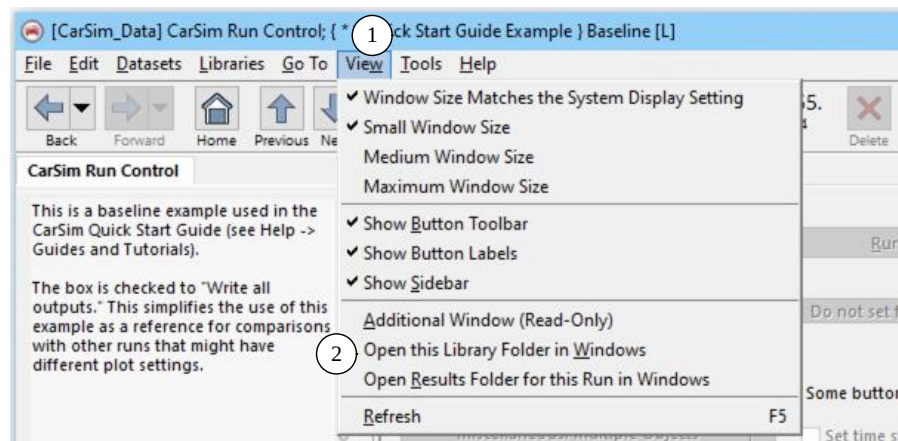
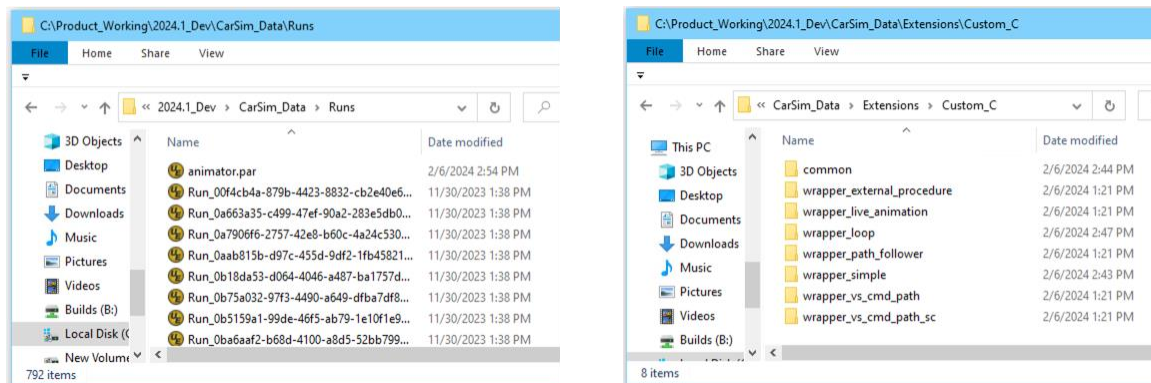


Figure 3. Select the View menu option Open this Library Folder in Windows.



a. Runs folder

b. Extensions\Custom C folder

Figure 4. User Windows File Explorer to locate the Extensions folder in the database.

Navigate to the database folder (e.g., CarSim_Data in the figure) and go to the folder Extensions (Figure 4b), which in turn contains folders for different languages (e.g., Custom_C).

Adding the Controller with VS Commands

This section describes how the controller is added to a math model using VS Commands. This is not a reference for VS Commands (that is the *VS Commands Reference Manual*); it is introductory in nature. Readers who are familiar with VS Commands and are mainly interested in the API examples may skim this section or even skip to the next one.

VS Commands for the Example

The example run for this example in CarSim and TruckSim is { **Extended Models** } **Steer Controller, VS Commands**. In BikeSim it is { **Extended Models** } **Lean Controller, VS Commands**.

In reviewing the equations from the preceding section, we see that they involve four existing output variables (T, Xo, Yo, and Yaw), two new variables (Xpreview and Ypreview), and three new parameters (LAT_TRACK, GAIN_PATH_CTRL, and L_FORWARD). The new variables Xpreview and Ypreview that are used to calculate the steering can also be helpful if available for plotting, and for animating to show the preview point (see the arrow in Figure 1).

For this example, the VS Commands are placed in a dataset in the library **Generic: VS Commands** (Figure 5), and a link is made to this dataset from the **Run Control** screen.

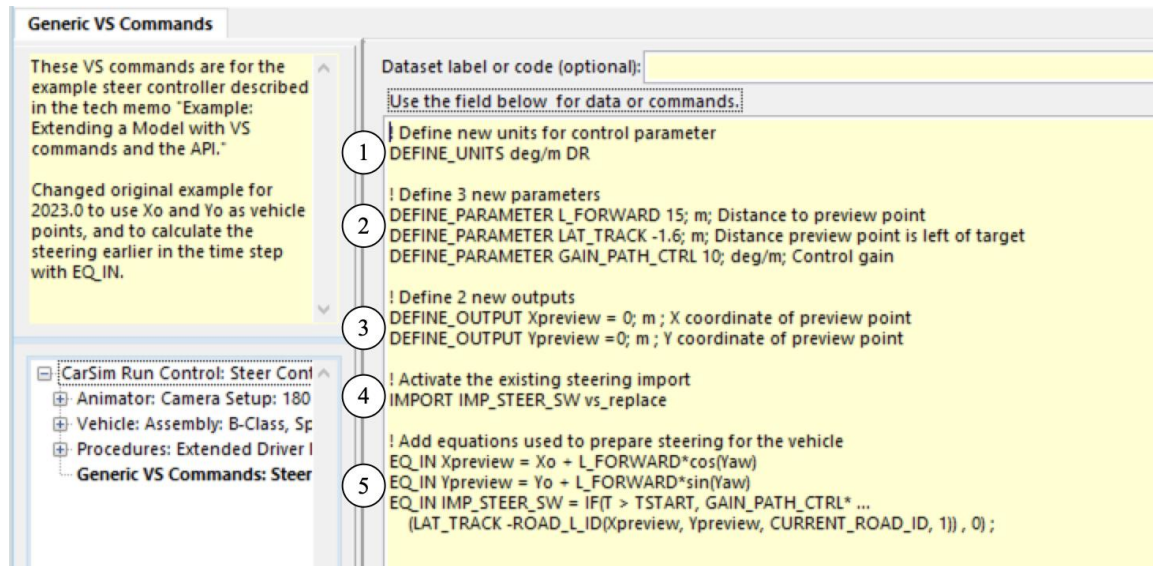


Figure 5. VS Commands for the example path follower.

The three new parameters are added with the command `DEFINE_PARAMETER` (2). The new output variables are defined with the command `DEFINE_OUTPUT` (3), and equations 1 and 2 are applied with the `EQ_IN` command (5).

All named parameters and variables in a VS Math Model have associated units. As described in the *VS Commands Reference Manual*, conversions are made automatically between *user units* shown for input and output files, and *internal dynamical units* used for internal calculations. For example, steering wheel angle has user units of degrees, and internal units of radians. All input angles are automatically divided by $180/\pi$ to convert from degrees to radians. For output, the angles are automatically multiplied by $180/\pi$ to convert from radians to degrees.

Existing user units are shown in the readme files and the Echo files generated with each run.

Note	The internal units are generally SI and require no scale factors for internal calculations. The parameter <code>OPT_ECHO_ALL_UNITS</code> can be set to any nonzero value to have the VS Math Model show everything about the current user unit system in the Echo file, including all of the scale factors that relate user units to internal units.
-------------	---

Most of the variables and parameters that will be added for this example involve length and are assigned units of meters (meters are used for most of the road geometry). The exception is the constant `GAIN_PATH_CTRL` (2), which should have units of steer or lean angle per unit of lateral distance: deg/m.

If a run is made with the `DEFINE_PARAMETER` command for `GAIN_PATH_CTRL`, without preparation, an error message appears saying the units “deg/m” are not installed. To allow the use of “deg/m,” the `DEFINE_UNITS` command (1) is used to install everything the solver needs to know about the new units: the printed name is deg/m, and the scale factor is set to a built-in constant `DR` (degrees/radian) = $180/\pi \approx 57.29577951$. After the new units are installed, they can be specified with the keyword `DEG/M` (not case sensitive), as shown in the `DEFINE_PARAMETER` command for `GAIN_PATH_CTRL` (2).

The `IMPORT` command (4) activates the imported steering wheel angle with a copy of equation 3. The third `EQ_IN` command (5) covers two lines. The first line ends with the “...” continuation indicator. When the VS Math Model scans the line, it finds the three consecutive dots. It then clears the contents of the internal copy of the text from the location of the first dot and continues with the first non-blank character in the next line (starting with “(LAT_TRACK” in this example).

At this point (after statement (5)), the controller is fully defined, and the additional X and Y output coordinates are available for plotting and animation.

The VS Commands are shown in the Echo files generated by the solver programs when runs are made. Most commands appear at the end of the file (Figure 6). The printed version in the Echo file is not necessarily identical in appearance to the original commands (Figure 5), but it has the same meaning and will give identical results if used in future runs. One difference is that all variables in equations are shown in upper case, regardless of how they were provided as input.

Note	Output variables such as <code>Xpreview</code> are represented internally with two names for a single variable. One is the keyword used to identify any reference to the variable in equations. The keyword is uppercase (e.g., <code>XPVIEW</code>), as are the keywords for all other variables in the model. The
-------------	--

second name, with the original mixed case (e.g., Xpreview) is written into the output VS or ERD file where it is available to support automatic labeling of plots.

```

5886 !-----
5887 ! NEW VARIABLES DEFINED AT RUN TIME
5888 !-----
5889 DEFINE_PARAMETER L_FORWARD = 15; m ; Distance to preview point
5890 DEFINE_PARAMETER LAT_TRACK = -1.6; m ; Distance preview point is left of target
5891 DEFINE_PARAMETER GAIN_PATH_CTRL = 10; deg/m ; Control gain
5892
5893 DEFINE_OUTPUT Xpreview = 100.091; m ; X coordinate of preview point
5894 DEFINE_OUTPUT Ypreview = -56.6283; m ; Y coordinate of preview point
5895
5896 !-----
5897 ! EQUATIONS IN (BEFORE DRIVER CONTROLS)
5898 !-----
5899 EQ_IN XPREVIEW = XO + L_FORWARD*COS(YAW);
5900 EQ_IN YPREVIEW = YO + L_FORWARD*SIN(YAW);
5901 EQ_IN IMP_STEER_SW = IF(T > TSTART, GAIN_PATH_CTRL*(LAT_TRACK -ROAD_L_ID(XPREVIEW,
5902 YPREVIEW, CURRENT_ROAD_ID, 1)), 0);
5903
5904 !-----
5905 ! IMPORTED VARIABLES, RELATIONS TO NATIVE VARIABLES, INITIAL VALUES, and UNITS
5906 !-----
5907 IMPORT IMP_STEER_SW VS_REPLACE 0 ; deg ! #0. Steering wheel angle
5908
5909 END

```

Figure 6. Most VS Commands are written at the end of an Echo file.

Whenever the unit system of the VS Math Model is modified (e.g., adding the new units “deg/m”), a set of commands appears at the top of the Echo file to fully specify the unit system as it was modified in the run (line 16, Figure 7).

```

1 PARSFILE
2 ! CarSim 2023.0
3 ! Revision 191843, November 20, 2022
4 MODEL_LAYOUT I_I
5
6 DATASET_TITLE Path Follower, VS Commands, 2023
7 CATEGORY Extended Models
8 TITLE Path Follower, VS Commands, 2023 <Extended Models>
9
10 ! Echo: Results\Run_06802b89-a825-4d43-ba60-23856702b9a8\LastRun_echo.par
11 ! This run was made 09:11 on November 21, 2022.
12
13 !-----
14 ! CURRENT UNIT-SYSTEM KEYWORDS, WRITTEN DESCRIPTIONS, AND SCALE FACTORS
15 !-----
16 DEFINE_UNITS deg/m 57.2957795131
17

```

Figure 7. DEFINE_UNITS command written near the top of an Echo file.

Running the Extended Model

The VS Commands can be placed anywhere in the VS database where they would be passed to the math model when it reads inputs for a new run. A simple option is to link to the VS Command dataset from the **Run Control** screen, as was done for this example.

No other steps are needed to apply the model extensions. Use the regular **Run Math Model** button to make the run, and view results with the regular **Video** and **Plot** buttons. For example, Figure 8 shows the video and two plots from an example run in CarSim. Note that the top plot shows the trajectory of two points of interest, including the preview point, identified in the plot by the label `Ypreview`. The video also shows the preview point with a yellow arrow.

Extending a VS Model with a Custom Wrapper: Overview

The tech memo *Automating Runs with the VS API* explains how to load and run a VS Math Model using a VS Solver library used by external software. This method can be extended to include the steering controller as programmed using MATLAB and C/C++ in the following sections. In each case, a short program will load the VS Solver library and make the run. A program such as this that loads and runs a dynamic library is called a *wrapper*.

In normal operation on a Windows-based PC, the top-level input file for a VS Math Model is a simulation control file that contains pathnames for the input and output files that will be handled by the solver, plus the pathname for the VS Solver DLL. This is called the *Simfile* and has the default name `simfile.sim`. Details of Simfile contents are provided in the *VS Math Models* reference manual.

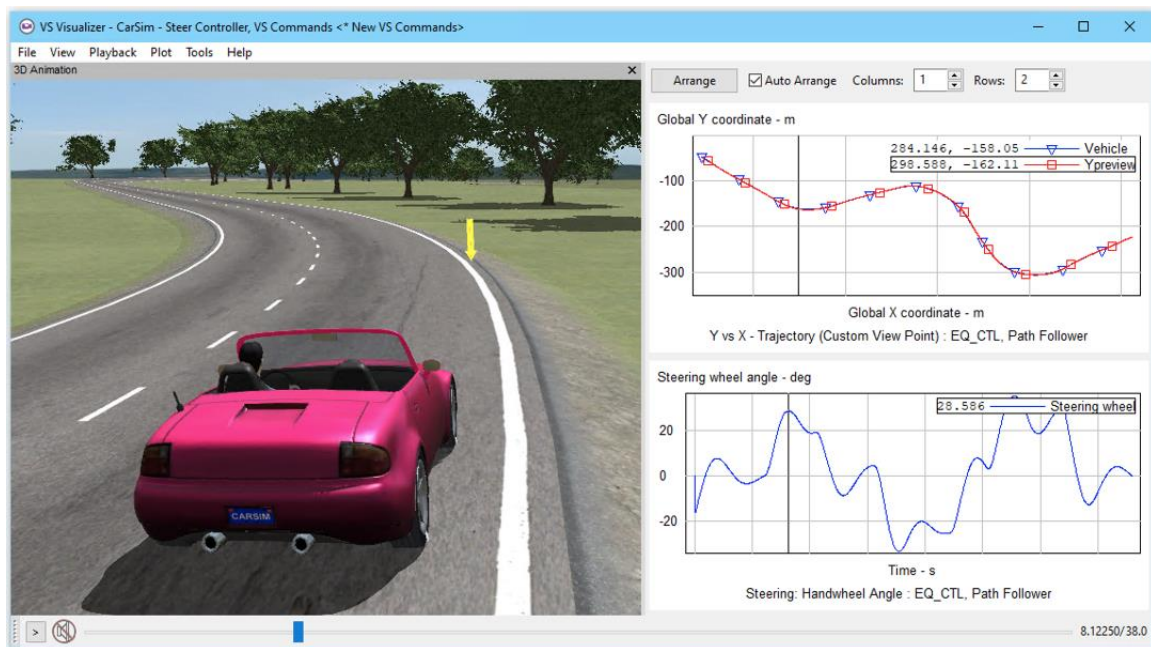


Figure 8. Results obtained with the example custom steering controller.

Making a run with the VS Solver involves these steps:

1. **Load.** Get the VS Solver library pathname from a Simfile, load library, and access the VS API functions contained in the library.
2. **Initialize.** Read the Simfile (again) to obtain pathnames for input and output files that will be used for the run. Read input data and create and initialize a VS Math Model, whose layout is defined by input Parsfiles.

3. **Run.** Run the simulation with a simple loop in which time advances in small increments. The VS Math Model updates variables at each timestep.
4. **Terminate.** Finish files that were updated during the run and free memory taken by the VS Math Model.
5. **Unload.** Free the memory used by the library.

Full details of this process are covered in the *VS API Manual*.

In the examples described in the tech memo *Automating Runs with the VS API*, the only critical VS API function used in each example wrapper program is `vs_run`, which performs steps 2 through 4 above. When a wrapper program is used to extend the model, then it is usually necessary to perform steps 2 to 4 separately. Typically, new inputs are added programmatically as part of the initialization, and additional calculations are performed each timestep.

Extending a VS Model with MATLAB

VS Math Models are ready to run from within MATLAB/Simulink using VS S-Functions, and many users extend the models using Simulink with no knowledge or concern about the API. Alternatively, VS Math Models can be built and run directly from MATLAB (without Simulink) by using the API.

Note If you do not use MATLAB and are interested in making a C wrapper, you can skip ahead to the next section (page 21).
--

Review: Running a VS Math Model using MATLAB

The tech memo *Automating Runs with the VS API* describes an example of running a VS Math Model from MATLAB, which will be reviewed in this subsection.

Running a VS Math Model from MATLAB is usually done with a text MATLAB M file.

When running from MATLAB, the working directory will be the folder containing the MATLAB M file. Therefore, the Simfile should be written to the folder containing the M file. This is specified using a dataset from the library **Models: Transfer to Local Windows Directory** (Figure 9). The linked dataset should identify the folder containing the MATLAB file ①. In the same dataset, use the drop-down control for choosing a 32/64 bit solver ② to match your version of MATLAB. The examples shipped in VehicleSim products also specify a custom Simfile name, using the keyword `simfile` ③. For the examples in which no extensions are made to the VS Math Model, the Simfile is named `simple.sim`.

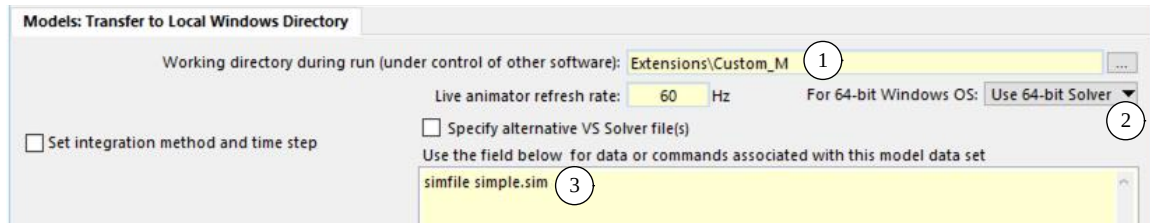


Figure 9. Specify the folder containing MATLAB files.

To use these settings, a link is made from the **Run Control** screen (Figure 10) to the dataset in the library **Models: Transfer to Local Windows Directory** (4), selected with the **Models** drop-down control (3). The linked dataset (2) is the one shown in Figure 9. Generate the files for the MATLAB program by clicking the button **Generate Files for this Run** (1).

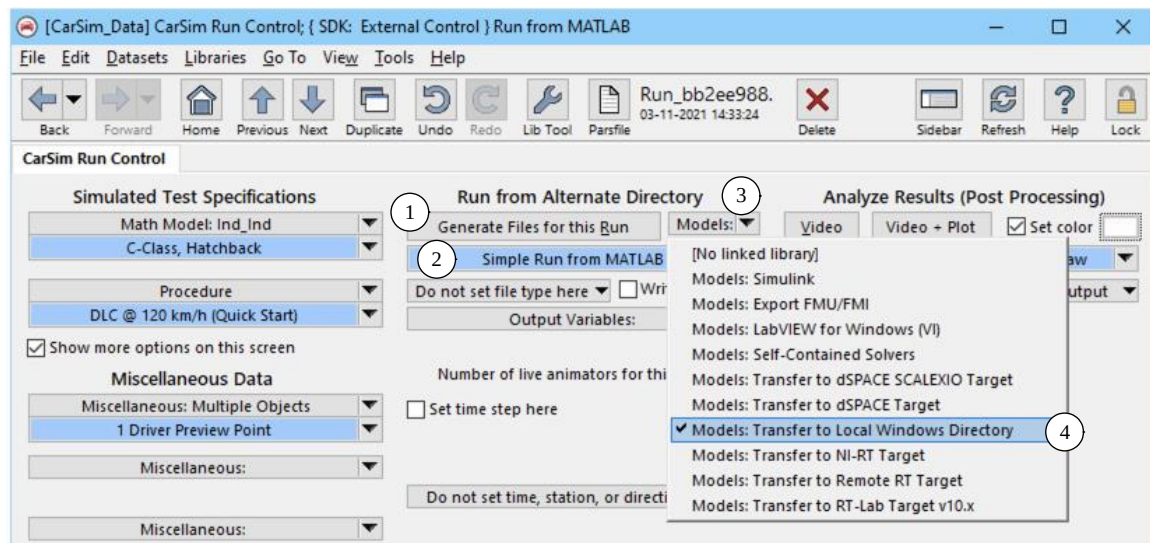


Figure 10. Link to the library Models: Transfer to Local Windows Directory.

To run the simulation from MATLAB (Figure 11), load the `simple.m` file (1) into MATLAB (2) and run it using a menu control, MATLAB command, or **Run** button (3) (the details for how you control MATLAB depend on the version). Status of the simulation is shown in the **Command Window** (4).

In this example, line 18 reads the simfile to obtain the pathname for the VS Solver DLL. Lines 21 and 22 load the solver, line 26 runs the simulation, and line 30 unloads the Solver DLL.

Running a Simulation Loop in MATLAB

Another M file `simple_loop.m` (Figure 12) performs the same purpose as `simple.m` but uses a few more commands to access the VS Math Model at different times in the simulation.

Both M files have the same code at the start and finish. At the start, both have the same commands going to `loadlibrary` (lines 23 and 24, Figure 12). Both finish with the `unloadlibrary` command (line 47 in Figure 12).

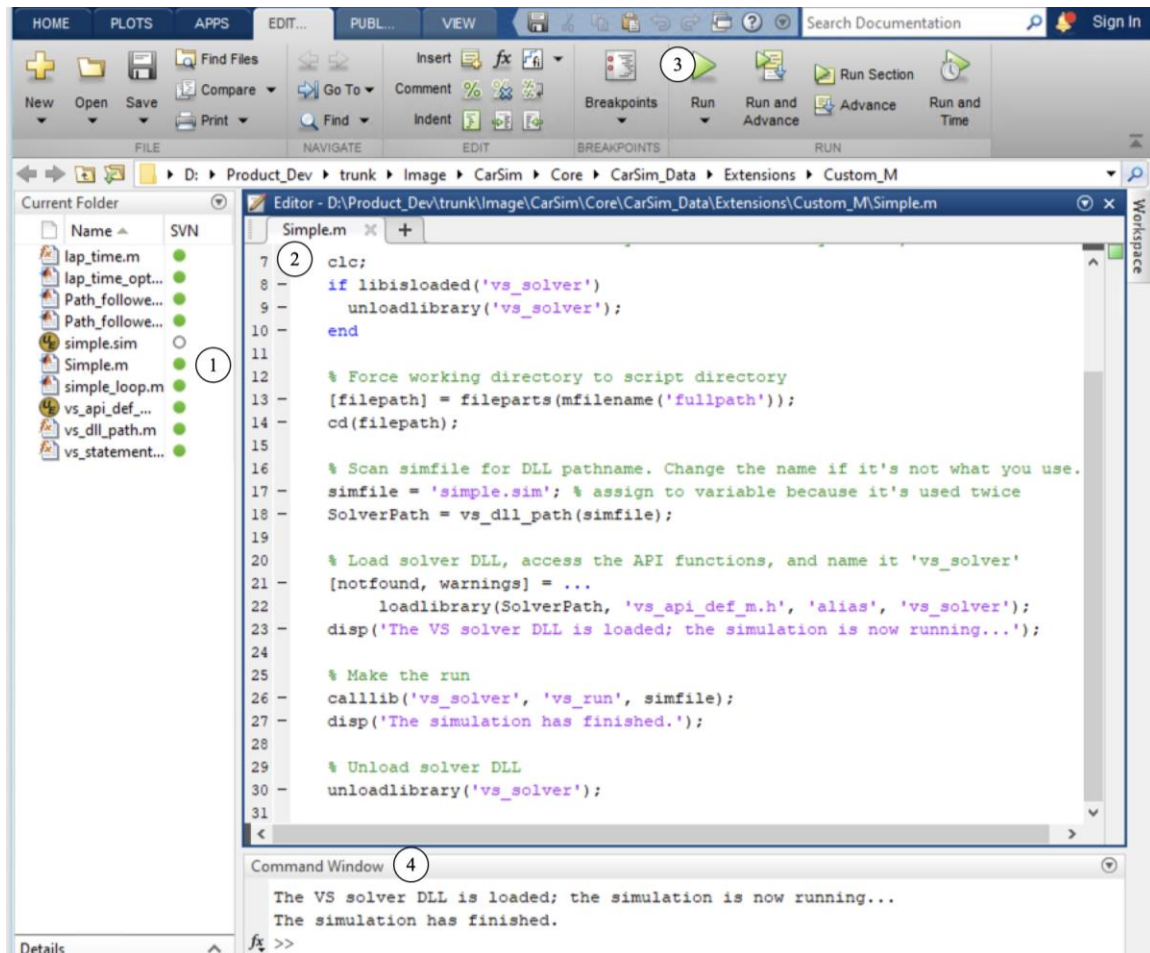


Figure 11. Running a VS Math Model using the `vs_run` function from MATLAB.

The difference is that in `Simple.m`, a single command runs the simulation using the VS API function `vs_run` via the MATLAB function `calllib`, whereas in `simple_loop.m` (Figure 12) the simulation is done in several steps. After the DLL is loaded, the model is initialized with four API function calls:

1. The API function `vs_setdef_and_read` (line 29) builds a VS Math Model and commands it to read all input Parsfiles. The function also returns the start time, which is assigned to a MATLAB variable `t`.
2. The API function `vs_initialize` (line 30) instructs the VS Math Model to calculate initial conditions based on all the information read from the input Parsfiles
3. The API function `vs_get_output_message` (line 31) provides text that is displayed in the MATLAB Command Window (4).
4. The API function `vs_error_occurred` (line 34) is used to see if any errors occurred during the initialization. The MATLAB variable `stop` is set to zero if no error occurred.

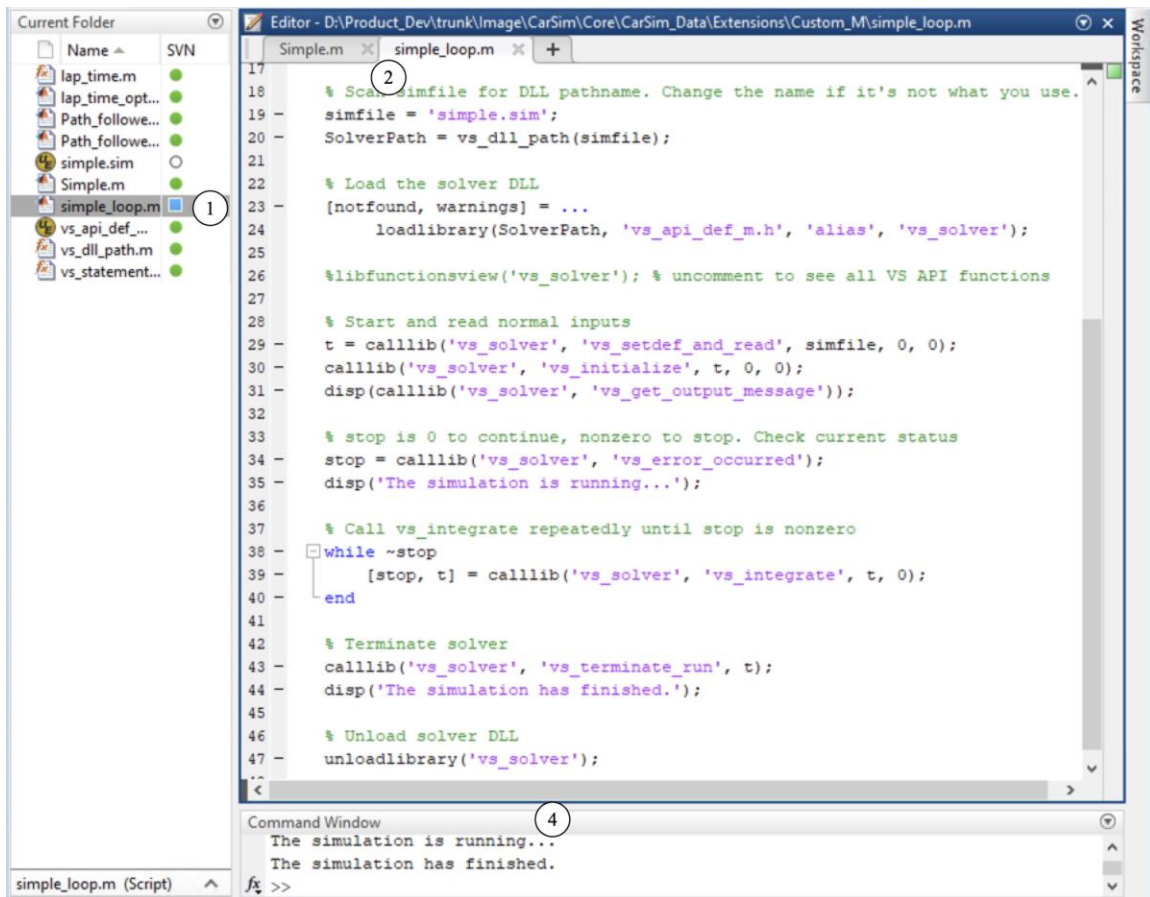


Figure 12. The *simple_loop* M-File has a structure suited for extending models.

The simulation is run with a repeated call to the `vs_integrate` function (line 39). Each timestep, the function updates two variables in MATLAB: `stop` and `t`. The simulation continues if `stop` remains zero. The current simulation time `t` is required as an input to `vs_integrate`. The variable `t` is also an output that is updated by `vs_integrate`.

When the simulation ends (`stop` is no longer zero), the API function `vs_terminate_run` is called to terminate the run (line 43). This cleans up memory and closes output files before the DLL is unloaded (line 47).

As noted earlier, the results obtained using `simple_loop.m` are identical to those obtained with `Simple.m`, which are in turn identical to running the VS Math Model directly from the VS Browser without any involvement of MATLAB.

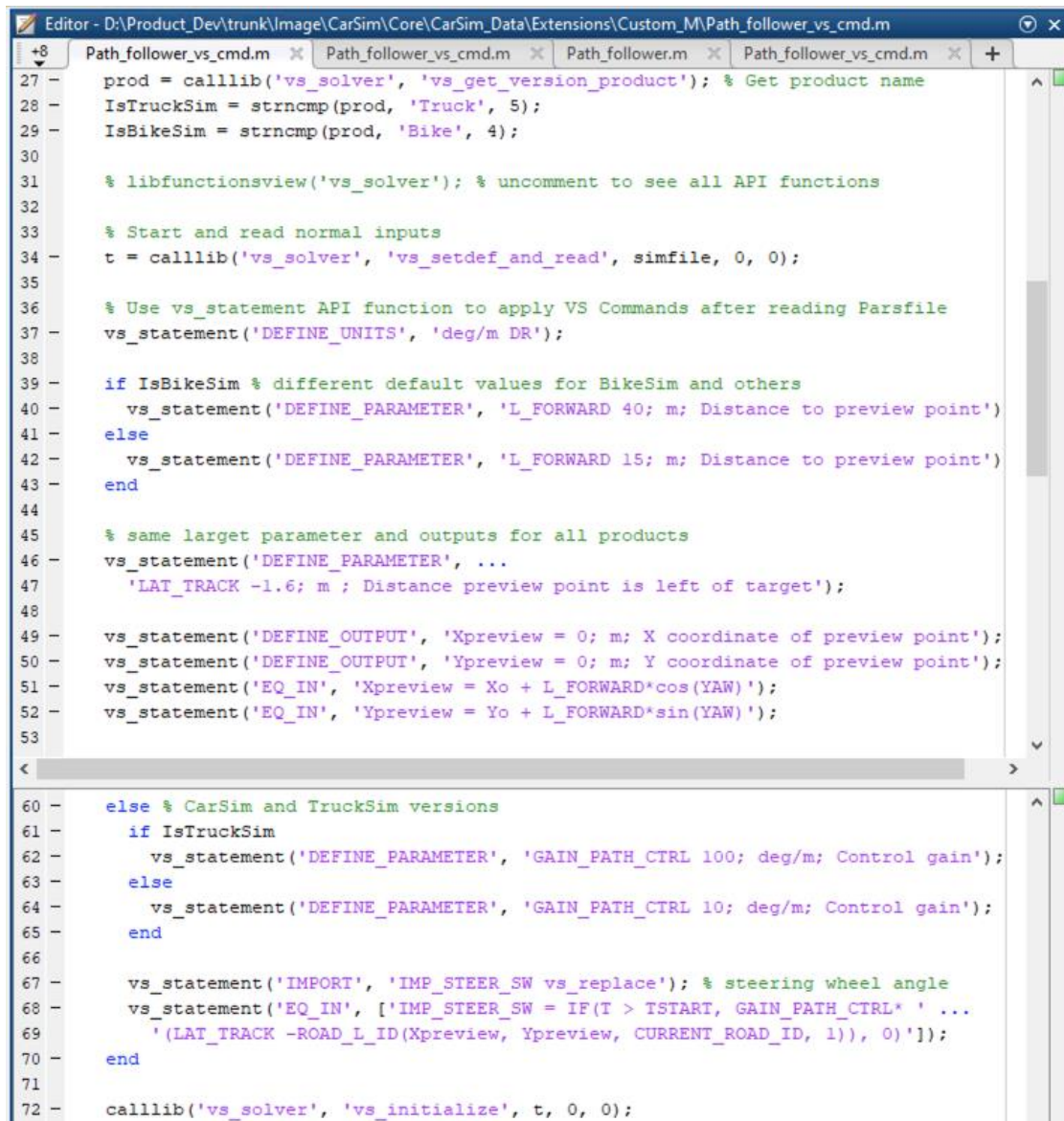
The reason for showing and explaining the layout of `simple_loop.m` is that this program will be extended in two following examples that show methods for adding the path follower to the vehicle model.

Applying VS Commands from MATLAB

There are situations where automation provided with MATLAB might be combined with standard inputs for the VS Math Model. A VS API function `vs_statement` can be used to provide almost

any input to the VS Math Model that can be provided from a Parsfile input as a single line of text. The only exceptions are tables of numbers, which require multiple lines of text.

For example, the VS Commands shown earlier to add the path follower to a VS Math Model are specified within a MATLAB file `Path_follower_vs_cmd.m` (Figure 13). This file was made by modifying the M file `simple_loop.m` shown earlier (Figure 12). In comparing the two files, you can see that the `simple_loop.m` file has a call to `vs_setdef_and_read` (line 29, Figure 12) immediately followed by a call to `vs_initialize` (line 30). In the `Path_follower_vs_cmd.m` file, VS Commands were inserted in between those existing statements (lines 34 and 72, Figure 13).



```

Editor - D:\Product_Dev\trunk\Image\CarSim\Core\CarSim_Data\Extensions\Custom_M\Path_follower_vs_cmd.m
+8 Path_follower_vs_cmd.m Path_follower_vs_cmd.m Path_follower.m Path_follower_vs_cmd.m
27 - prod = calllib('vs_solver', 'vs_get_version_product'); % Get product name
28 - IsTruckSim = strcmp(prod, 'Truck', 5);
29 - IsBikeSim = strcmp(prod, 'Bike', 4);
30
31 % libfunctionsview('vs_solver'); % uncomment to see all API functions
32
33 % Start and read normal inputs
34 - t = calllib('vs_solver', 'vs_setdef_and_read', simfile, 0, 0);
35
36 % Use vs_statement API function to apply VS Commands after reading Parsfile
37 - vs_statement('DEFINE_UNITS', 'deg/m DR');
38
39 if IsBikeSim % different default values for BikeSim and others
40     vs_statement('DEFINE_PARAMETER', 'L_FORWARD 40; m; Distance to preview point')
41 else
42     vs_statement('DEFINE_PARAMETER', 'L_FORWARD 15; m; Distance to preview point')
43 end
44
45 % same target parameter and outputs for all products
46 - vs_statement('DEFINE_PARAMETER', ...
47     'LAT_TRACK -1.6; m ; Distance preview point is left of target');
48
49 - vs_statement('DEFINE_OUTPUT', 'Xpreview = 0; m; X coordinate of preview point');
50 - vs_statement('DEFINE_OUTPUT', 'Ypreview = 0; m; Y coordinate of preview point');
51 - vs_statement('EQ_IN', 'Xpreview = Xo + L_FORWARD*cos(YAW)');
52 - vs_statement('EQ_IN', 'Ypreview = Yo + L_FORWARD*sin(YAW)');
53
54
55
56
57
58
59
60 - else % CarSim and TruckSim versions
61 -     if IsTruckSim
62 -         vs_statement('DEFINE_PARAMETER', 'GAIN_PATH_CTRL 100; deg/m; Control gain');
63 -     else
64 -         vs_statement('DEFINE_PARAMETER', 'GAIN_PATH_CTRL 10; deg/m; Control gain');
65 -     end
66
67     vs_statement('IMPORT', 'IMP_STEER_SW vs_replace'); % steering wheel angle
68     vs_statement('EQ_IN', ['IMP_STEER_SW = IF(T > TSTART, GAIN_PATH_CTRL* ' ...
69         '(LAT_TRACK -ROAD_L_ID(Xpreview, Ypreview, CURRENT_ROAD_ID, 1)), 0)']);
70 - end
71
72 - calllib('vs_solver', 'vs_initialize', t, 0, 0);

```

Figure 13. Adding the simple path follower with VS Commands from MATLAB.

Compare the display of the VS Commands at the end of the Echo file shown earlier (Figure 6, page 9) with the inputs to the `vs_statement` MATLAB function (lines 42 – 69, Figure 13).

This example M file works with CarSim, TruckSim, and BikeSim, using slightly different VS Commands for the different products. The VS API function `vs_get_version_product` is used to get a string identifying the product and version (line 27). This string, assigned to the variable `prod`, will begin with the text `CarSim`, `TruckSim`, or `BikeSim`. Two variables are set to 0 or 1 using the MATLAB function `strncmp` to compare the first 4 or 5 characters of the variable `prod` to the strings `"Truck"` and `"Bike"` (lines 28 and 29). If the product is `TruckSim`, then `IsTruckSim = 1`; if the product is `BikeSim`, then `IsBikeSim = 1`. If not, they are set to 0. If both are 0, the product is assumed to be `CarSim`.

These two Boolean variables are used to activate blocks of VS Commands. If `IsBikeSim` is not zero, then line 40 and lines 55-59 are evaluated (not shown). Otherwise, statements in lines 42 and 61-69 are evaluated. Lines 61-65 distinguish between `CarSim` and `TruckSim` using different default values for the parameter `GAIN_PATH_CTRL` for `TruckSim` (100 deg/m) or `CarSim` (10 deg/m).

When a simulation is run using the `Path_follower_vs_cmd.m` file, the resulting Echo file is nearly identical to the Echo file generated without MATLAB (e.g., Figure 6, page 9). Either way, the VS Math Model receives the same VS Commands. Without MATLAB, it was receiving the information from the datasets in the database (Figure 5, page 7). With MATLAB, it was receiving the information after all Parsfile information had been scanned (`vs_setdef_and_read`) but before the initialization was performed (`vs_initialize`).

Note	Functions from a DLL are applied using the MATLAB <code>calllib</code> function (see lines 27, 34, and 72 in Figure 13). A statement using <code>calllib</code> needs its own name (<code>calllib</code>), the name of the loaded DLL (<code>vs_solver</code>), the name of the DLL function (e.g., <code>setdef_and_read</code>), and additional arguments that are needed for the DLL function, as indicated in the documentation or the window made via the MATLAB <code>libfunctionsview</code> function.
	For this example, a MATLAB M-file function was made named <code>vs_statement</code> that in turn calls the <code>vs_statement</code> function from the DLL, with just two arguments (keyword and statement string). This was done for the purpose making the code shorter and more readable, given the multiple VS Commands.

The simulation results obtained with this MATLAB M-file are identical to those obtained using VS Commands entered via Parsfiles. This can be tested by comparing the End Echo Parsfile for this example runs with the example shown earlier. The only differences are in the titles and the `RTIME` calculated run-time ratio; everything else is identical in the two End Parsfiles, including the values of all state variables listed at the end of the files.

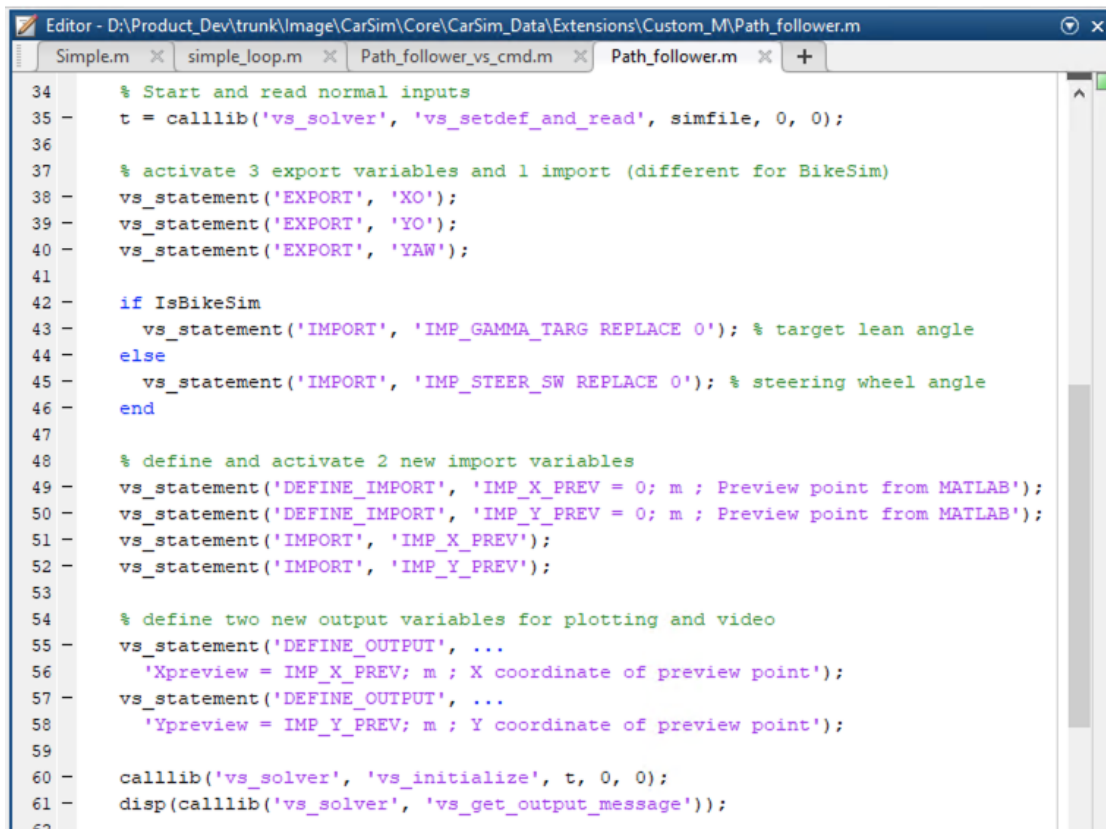
Using Import and Export Arrays with MATLAB

When running with Simulink, information is shared between the Simulink model and the VS Math Model using arrays of Import and Export variables. These arrays are always available for use in VS Math Models for sharing information with external programs, and may be used with MATLAB,

even without Simulink. Using the Import and Export arrays, the equations used for the example steering controller can be applied within MATLAB. To do this:

1. Arrays must be defined in MATLAB whose elements are the variables of interest.
2. Import and Export variables must be activated in the VS Math Model to match the definitions from MATLAB.
3. Every timestep, values for the array elements imported to the VS Math Model must be calculated in MATLAB, and the contents of the arrays must be shared between MATLAB and the VS Math Model.

The approach shown in the previous subsection for using VS Commands is again used in the example file `Path_follower.m`. Figure 14 shows that some VS Commands are again applied after the call to `vs_setdef_and_read` (line 35) and before the call to `vs_initialize` (line 63). As in the previous example, variables `IsBikeSim` and `IsTruckSim` are created allow the same M file to work with CarSim, TruckSim, and BikeSim.



```
34 % Start and read normal inputs
35 t = calllib('vs_solver', 'vs_setdef_and_read', simfile, 0, 0);
36
37 % activate 3 export variables and 1 import (different for BikeSim)
38 vs_statement('EXPORT', 'XO');
39 vs_statement('EXPORT', 'YO');
40 vs_statement('EXPORT', 'YAW');
41
42 if IsBikeSim
43     vs_statement('IMPORT', 'IMP_GAMMA_TARG REPLACE 0'); % target lean angle
44 else
45     vs_statement('IMPORT', 'IMP_STEER_SW REPLACE 0'); % steering wheel angle
46 end
47
48 % define and activate 2 new import variables
49 vs_statement('DEFINE_IMPORT', 'IMP_X_PREV = 0; m ; Preview point from MATLAB');
50 vs_statement('DEFINE_IMPORT', 'IMP_Y_PREV = 0; m ; Preview point from MATLAB');
51 vs_statement('IMPORT', 'IMP_X_PREV');
52 vs_statement('IMPORT', 'IMP_Y_PREV');
53
54 % define two new output variables for plotting and video
55 vs_statement('DEFINE_OUTPUT', ...
56     'Xpreview = IMP_X_PREV; m ; X coordinate of preview point');
57 vs_statement('DEFINE_OUTPUT', ...
58     'Ypreview = IMP_Y_PREV; m ; Y coordinate of preview point');
59
60 calllib('vs_solver', 'vs_initialize', t, 0, 0);
61 disp(calllib('vs_solver', 'vs_get_output_message'));
62
```

Figure 14. Setting up Import, Export, and Output variables for the VS Math Model.

The vehicle Yaw and X and Y coordinates are needed to calculate a control angle (steer or lean) in MATLAB and must therefore be exported. These variables are activated for Export with three EXPORT VS Commands.

The control to be imported differs for BikeSim (line 43) and the others (line 45).

For this example, the calculations of the global X and Y coordinates of the preview point are performed in MATLAB. To use these values in VS Visualizer for plotting and animation, they must be imported from MATLAB, and assigned to output variables in the VS Math Model.

To do this, two new Import variables (`IMP_X_PREV` and `IMP_Y_PREV`) are defined with the `DEFINE_IMPORT` VS Command (lines 49 and 50). These are activated for Import with the `IMPORT` command (lines 51 and 52).

Two new output variables are defined with the same names as before (`Xpreview` and `Ypreview`, lines 55 - 58). However, in this case, the formula for each is simply the symbol for the corresponding Import variable (`IMP_X_PREV` and `IMP_Y_PREV`, respectively).

To summarize: the code shown in Figure 14 sets up the VS Math Model to export three variables each timestep (`XO`, `YO`, and `YAW`) and to import three variables each timestep (`IMP_X_PREV`, `IMP_Y_PREV`, and a control angle).

Figure 15 shows the continuation of `Path_follower.m`. In this example, the parameters used to calculate steering wheel angle are MATLAB variables that are assigned values (lines 64 – 67). Alternative values are set for `BikeSim` (lines 71 and 72) and `TruckSim` (line 74). The remaining lines of code apply for all three products.

To match the setup of the VS Math Model for an Import array with three variables and an Export array with three variables, two arrays are defined in MATLAB with three elements each (lines 78 and 79). Arrays are accessed with VS API functions using pointers, so a pointer to each array is defined (lines 80 and 81).

As in the previous examples, a `while` loop is used to perform calculations at each timestep (lines 90 - 110). In the other example, the loop contained a single MATLAB statement using the VS API function `vs_integrate` (line 39, Figure 12, page 14). However, in this case, the loop contains more statements, needed to make use of variables exported from the VS Math Model to calculate the three variables that will be imported to the VS Math Model. In support of the transfer of variables using arrays, a different API function is used (`vs_integrate_io`, line 109), with additional arguments.

Each timestep, two actions are needed in MATLAB to connect the values within the arrays to the pointer variables used to communicate with the VS Math Model.

1. Values from the VS Math Model are copied to MATLAB using the `get` function (line 94).
2. The pointer sent to the VS Math Model for the Import array is updated each timestep using the `set` function (line 106).

In addition to the two pointers, the API function `vs_integrate_io` needs the current simulation time `t`. A statement at the top of the loop (line 91) updates `t` using the timestep, which was obtained after the initialization using the API function `vs_get_tstep` (line 84).

Each time through the loop, MATLAB needs the values of the three output variables exported by the VS Math Model. Before starting the loop, the pointer to the Export array from the VS Math Model is updated using the API function `vs_copy_export_vars` (line 85). Every timestep, the pointer is updated by `vs_integrate_io` (line 109).

```

63 % Define local parameters for the controller, with values set for CarSim
64 DR = 180/pi; % constant for units conversion: degrees per radians
65 Lfwd = 15; % preview distance
66 GainCtrl = 10; % control gain for steering or lean angle (deg/m)
67 LatTrack = -1.6; % projected lateral position relative to path
68
69 % adjust a few parameter values for BikeSim and TruckSim
70 if IsBikeSim
71     Lfwd = 40.0;
72     GainCtrl = -6;
73 elseif IsTruckSim
74     GainCtrl = 100;
75 end
76
77 % Define import/export arrays (both with length 3) and pointers to them
78 imports = zeros(1, 3);
79 exports = zeros(1, 3);
80 p_imp = libpointer('doublePtr', imports);
81 p_exp = libpointer('doublePtr', exports);
82
83 % get time step and export variables from the initialization
84 t_dt = calllib('vs_solver', 'vs_get_tstep');
85 calllib('vs_solver', 'vs_copy_export_vars', p_exp);
86 stop = calllib('vs_solver', 'vs_error_occurred');
87 disp('The simulation is running...');
88
89 % This is the main simulation loop. Continue as long as stop is 0.
90 while ~stop
91     t = t + t_dt; % increment time
92
93     % Update the array of exports using the pointer p_exp
94     exports = get(p_exp, 'Value');
95     Yaw = exports(3)/DR; % convert from deg to rad
96
97     % calculate values for MATLAB-based controller
98     Xpreview = exports(1) + Lfwd*cos(Yaw);
99     Ypreview = exports(2) + Lfwd*sin(Yaw);
100     road_l = calllib('vs_solver', 'vs_road_l', Xpreview, Ypreview);
101
102     % copy values into import array and set pointer for the VS solver
103     imports(1) = GainCtrl*(LatTrack - road_l);
104     imports(2) = Xpreview;
105     imports(3) = Ypreview;
106     set(p_imp, 'Value', imports); %set pointer for array of imports
107
108     % Call VS API integration function and exchange import and export arrays
109     stop = calllib('vs_solver', 'vs_integrate_io', t, p_imp, p_exp);
110 end
111
112 % Terminate
113 calllib('vs_solver', 'vs_terminate_run', t);
114 disp('Simulation has finished.');
```

Figure 15. Using arrays of Import and Export variables within MATLAB.

The X and Y coordinates for the preview point are calculated using the exported X and Y coordinates and vehicle yaw (lines 98 and 99). The lateral position of the preview point is calculated with the VS API equivalent of the ROAD_L function (line 100) and used to determine the control angle (steer or lean), which is the first value in the Import array (line 103). In addition, the X and Y coordinates of the preview point are sent to the VS Math Model using the second and third elements of the Import array (lines 104 and 105).

As with the other examples, the function `vs_terminate_run` is called (line 113) when the run finishes. The DLL is then unloaded (not shown in the figure).

This example is run using the same type of setup as the other MATLAB examples.

Recall that the MATLAB example shown in the previous subsection that applied VS Commands obtained identical results to the example made with VS Commands read from Parsfiles. However, in this example, the results are very close but not identical. If you compare the Echo Parsfile from this example (Figure 16) to the similar Echo file from the VS Command example, you will see some differences:

```

5891 ! -----
5892 ! NEW VARIABLES DEFINED AT RUN TIME
5893 ! -----
5894 DEFINE_IMPORT IMP_X_PREV = 0; m ; Preview point from MATLAB
5895 DEFINE_IMPORT IMP_Y_PREV = 0; m ; Preview point from MATLAB
5896
5897 DEFINE_OUTPUT Xpreview = 0; m ; X coordinate of preview point
5898 DEFINE_OUTPUT Ypreview = 0; m ; Y coordinate of preview point
5899
5900 ! -----
5901 ! EQUATIONS OUT (AFTER VEHICLE DYNAMICS)
5902 ! -----
5903 EQ_OUT XPREVIEW = IMP_X_PREV;
5904 EQ_OUT YPREVIEW = IMP_Y_PREV;
5905
5906 ! -----
5907 ! IMPORTED VARIABLES, RELATIONS TO NATIVE VARIABLES, INITIAL VALUES, and UNITS
5908 ! -----
5909 IMPORT IMP_STEER_SW REPLACE 0 ; deg ! #1. Steering wheel angle
5910 IMPORT IMP_X_PREV ADD 0 ; m ! #2. Preview point from MATLAB
5911 IMPORT IMP_Y_PREV ADD 0 ; m ! #3. Preview point from MATLAB
5912
5913 ! -----
5914 ! EXPORTED VARIABLES
5915 ! -----
5916 EXPORT XO 87.03582638 ! #1. X coordinate, vehicle origin (m)
5917 EXPORT YO -49.24190177 ! #2. Y coordinate, vehicle origin (m)
5918 EXPORT YAW -29.50039608 ! #3. Yaw, vehicle (deg)
5919

```

Figure 16. Echo file for MATLAB example using Import and Export arrays.

1. The VS Command version defines three new parameters (`L_FORWARD`, `LAT_TRACK`, and `GAIN_STEER_CTRL`) that are not echoed by the VS Math Model for the array version. (They are variables defined within MATLAB.)
2. Two new `IMPORT` variables (`IMP_X_PREV` and `IMP_Y_PREV`) are defined in the array version.
3. The VS Command version includes an `EQ_IN` equation for `IMP_STEER_SW` that is not part of the array version.
4. The `EQ_OUT` commands used to calculate the output variables `Xpreview` and `Ypreview` have full formulas in the VS Command version, but simply copy the values of Import variables in the array version.
5. The array version includes two `IMPORT` and three `EXPORT` commands that do not exist in the VS Command version.

When results from the two runs are overlaid in VS Visualizer, the vehicle motions are almost identical, but there are slight differences that will appear as “flicker” if the vehicles have different colors. The differences can also be seen by comparing the End Parsfiles from the two simulations. In addition to the differences in the VS Commands described above, the final values of the state variables in the VS Math Models are slightly different.

The slight differences in the state of the vehicle at the end of the run are due to differences in the timing of the calculations made with the two examples. The VS Commands perform all calculations within a timestep. The preview point coordinates X_{preview} and Y_{preview} are calculated at the beginning of each timestep and are used to calculate a control angle that is applied during the same timestep. On the other hand, with the array method described in this subsection, MATLAB receives the vehicle coordinates X_0 and Y_0 in an array exported by the VS Math Model the previous timestep. MATLAB calculates the control angle to be imported by the VS Math Model for the next timestep. That is, the array method inserts a delay of one timestep (typically 0.0005s) that can be avoided when VS Commands are used.

Extending a VS Model with C/C++

Methods for extending VS Math Models in high-level languages such as MATLAB can be also used in C. However, when running a VS Math Model from a wrapper program written in C/C++, several options are available that are not possible with MATLAB, because the wrapper can use C pointers to directly access variables and functions:

1. The wrapper program can directly access and possibly change any variable within the VS Math Model that has an identifying keyword; Import and Export arrays are not needed.
2. The wrapper program can install I/O support for new variables that exist in the wrapper, but which may be handled by the VS Math Model as parameters, output variables, state variables, etc.
3. The wrapper program can provide callback functions that are applied automatically by the VS Math Model at specified times within the simulation process. The callback functions can in turn access existing variables, add new variables, perform calculations, etc.

Review: Running a C Wrapper EXE Program

As described in the tech memo *Automating Runs with the VS API*, the **Run Control** dataset for a simulation made with a wrapper EXE should include a link (2) (Figure 17) to a dataset in the library **Models: Self-Contained Solvers** (4) selected with the **Models** drop-down control (3). With this setup, you can run a simulation by clicking the **Run Math Model** button (1), just as with the built-in VS Math Models.

In the linked **Models: Self-Contained Solvers** dataset (Figure 18), check the box to use an external wrapper program (1) and specify the pathname for the compiled wrapper program (2).

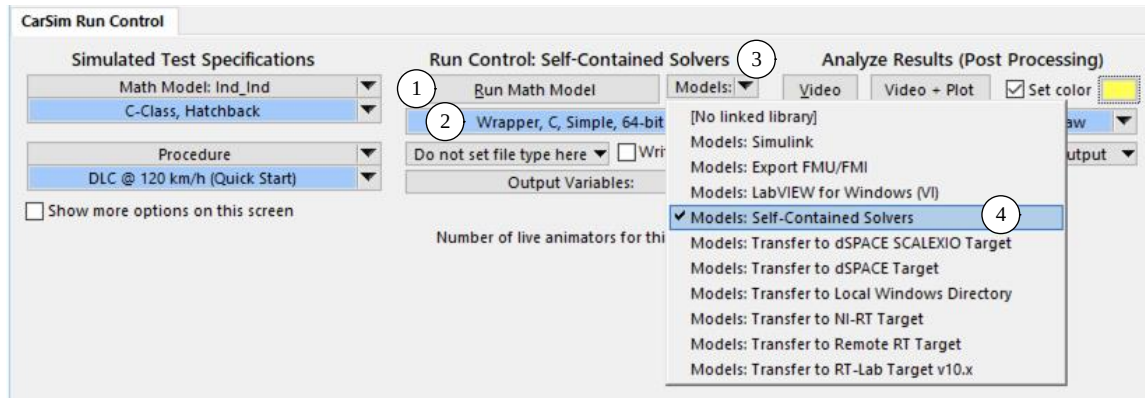


Figure 17. Link to the library Models: Self-Contained Solvers.

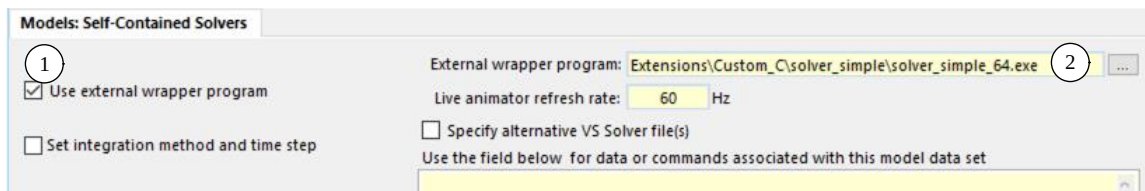


Figure 18. Identify the pathname to the compiled EXE wrapper program.

Applying VS Commands from a C Wrapper Program

The path follower was implemented using VS Commands in one of the earlier MATLAB examples, and the same approach is used with the C wrapper program contained in the folder `Extensions\Custom_C\wrapper_vs_cmd_path`.

Figure 19 shows the source code that handles some of the initialization part of the simulation. As seen in other examples, it uses a text string `simfile` to identify the Simfile (lines 34 and 38), uses a function `vs_get_dll_path` to scan the Simfile and determine the pathname for the VS Solver DLL (line 39), loads the VS Solver DLL (line 41), gets the VS API functions (line 47), and uses the API function `vs_setdef_and_read` to start the VS Math Model and have it read input Parsfiles (line 55).

This example also uses two local variables to handle differences between CarSim, TruckSim (line 51), and BikeSim (line 52).

The API function `vs_statement` is used to provide the same VS Commands that were shown earlier for a dataset managed by the VS Browser (Figure 5, page 7) and as specified in a MATLAB M file (Figure 13, page 15). The same statements are used in this C example (lines 59 – 72 show some of the VS Commands). When the wrapper program is run, the same results are obtained as were seen before in the simulation results and the appearance of the VS Commands at the end of the Echo file.

```

28  /*
29  Main program to run DLL with VS API.
30  */
31  int main(int argc, char** argv) {
32      HMODULE vsDLL = NULL; // DLL with VS API
33      double t; // simulation time
34      char pathDLL[FILENAME_MAX], simfile[FILENAME_MAX] = { "simfile.sim" };
35      int ibarg = 0; // counter for text bar graph
36
37      // get simfile from argument list and load DLL
38      if (argc > 1) strcpy(simfile, &argv[1][0]);
39      if (vs_get_dll_path(simfile, pathDLL)) return 1;
40      #if defined(_WIN32) || defined(_WIN64)
41      vsDLL = LoadLibrary(pathDLL);
42      #else
43      vsDLL = vs_load_library(pathDLL);
44      #endif
45
46      // get API functions
47      if (vs_get_api(vsDLL, pathDLL)) return 1;
48      sMsg = vs_get_output_message(); // pointer to text from DLL for sPrintMsg()
49
50      // is this not CarSim?
51      sTruckSim = strcmp(vs_get_version_product(), "Truck", 5) == 0;
52      sBikeSim = strncmp(vs_get_version_product(), "Bike", 4) == 0;
53
54      // Read normal inputs
55      t = vs_setdef_and_read(simfile, NULL, NULL);
56      if (vs_error_occurred()) return 1;
57
58      // VS Commands to add a controller with differences for BikeSim, CarSim, and TruckSim
59      vs_statement("DEFINE_UNITS", "deg/m DR", 1);
60
61      if (sBikeSim) // different default value for BikeSim
62      vs_statement("DEFINE_PARAMETER", "L_FORWARD 40; m ; Distance to preview point", 1);
63      else
64      vs_statement("DEFINE_PARAMETER", "L_FORWARD 15; m ; Distance to preview point", 1);
65
66      vs_statement("DEFINE_PARAMETER",
67      "LAT_TRACK -1.6; m ; Distance preview point is left of target", 1);
68      vs_statement("DEFINE_OUTPUT", "Xpreview = 0; m ; X coordinate of preview point", 1);
69      vs_statement("DEFINE_OUTPUT", "Ypreview = 0; m ; Y coordinate of preview point", 1);
70
71      vs_statement("EQ_IN", "Xpreview = XO + L_FORWARD*cos(YAW)", 1);
72      vs_statement("EQ_IN", "Ypreview = YO + L_FORWARD*sin(YAW)", 1);
73
74      // add parameters and commands customized for BikeSim
75      if (sBikeSim) {

```

Figure 19. C Wrapper Program that applies VS Commands.

Using Callback Functions from a C Wrapper Program

The example path follower is added to the model using a different approach with the C wrapper program from the folder Extensions\Custom_C\wrapper_path_follower. In this case, the model extension is defined using C callback functions. The parameters for the example path follower will not only be used in externally programmed equations, but they will be installed into the VS Math Model so new values can be set at runtime using the same VS GUI and database. In addition, the values for the parameters will be shown in the Echo files generated with each run.

Figure 20 shows C code for wrapper_extended.c. This is much simpler than the previous example; it is like the program described in the tech memo *Automating Runs with the VS API*. It also scans a specified Simfile and loads the DLL for the VS Solver (lines 28 and 30), gets the API functions from the DLL (line 35), and uses the API function `vs_run` to make the simulation and show a popup message if something goes wrong (lines 44 and 45). After handling a possible pause (lines 51 – 56), the memory used by the DLL is freed and the wrapper program exits.

However, unlike the simpler automation example, this wrapper installs externally defined functions that are deployed at various times when the VS Math Model runs (lines 38 – 40).

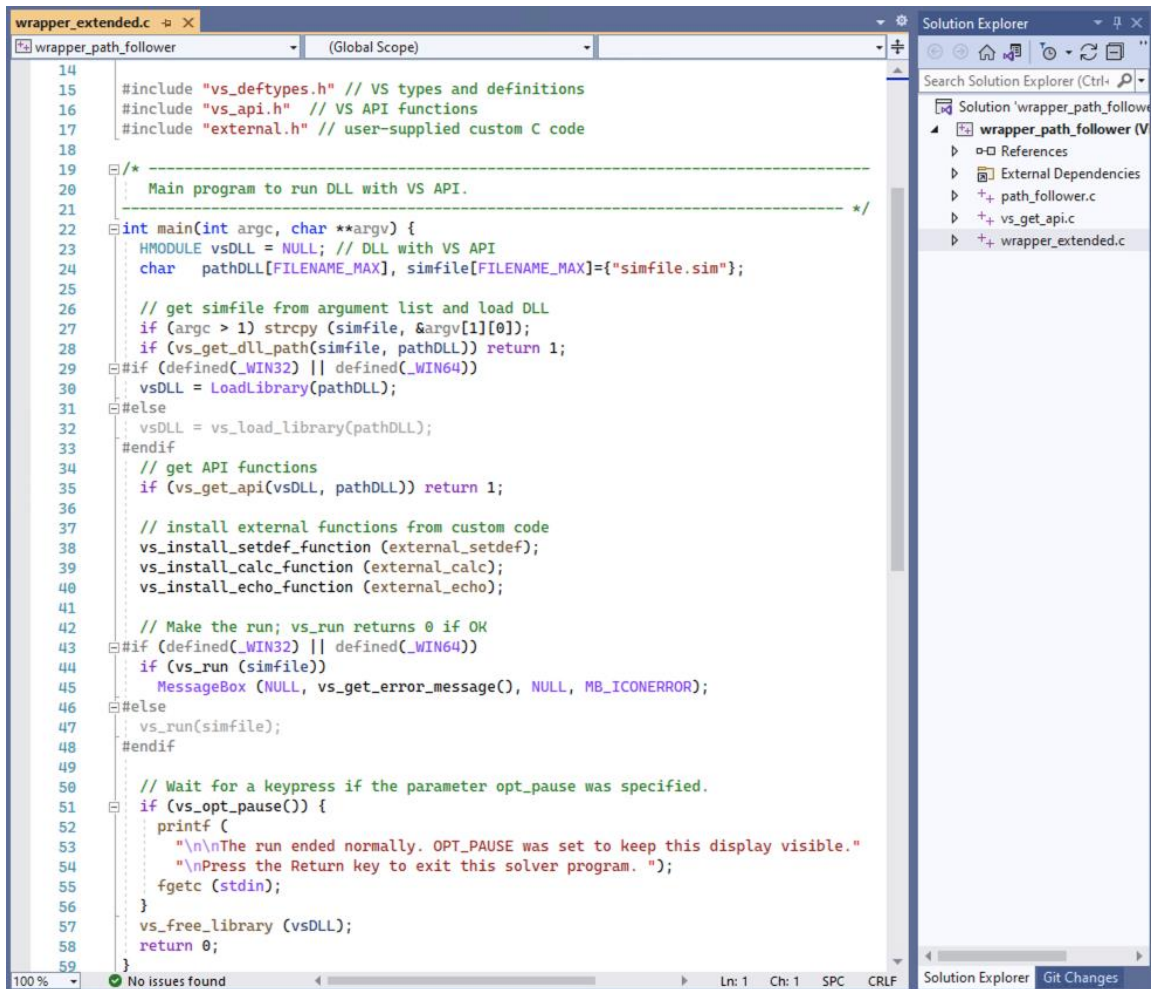


Figure 20. Main program for C wrapper that uses callback functions.

The file shown (wrapper_extended.c) is generic; the details of the model extension are provided by the three functions: external_setdef, external_calc, and external_echo. These are defined in a separate file named path_follower.c.

Figure 21 shows the top part of path_follower.c, with the #include statements and declarations of static variables used to support the example path follower (lines 21 - 30). Most of the variables are declared with type vs_real, which is normally set to the C type double in the header file vs_deftypes.h. There are also four integer variables: sUserExternal, sBikeSim, sTruckSim, and sCurRdId (lines 28 - 30).

Setting up the external code with external_setdef

Figure 22 shows part of the source code for external_setdef, the function that defines new variables and links to the variables in the VS Math Model. This is the name that was specified earlier using the API function vs_install_setdef_function (line 38, Figure 20).

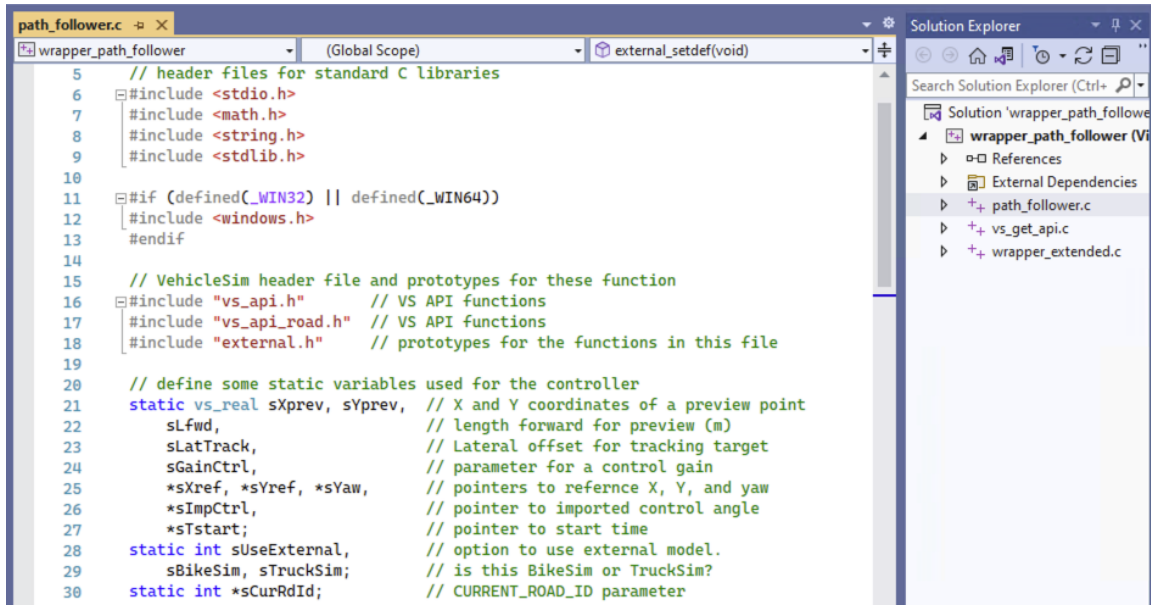


Figure 21. Required #include statements and shared static variables.

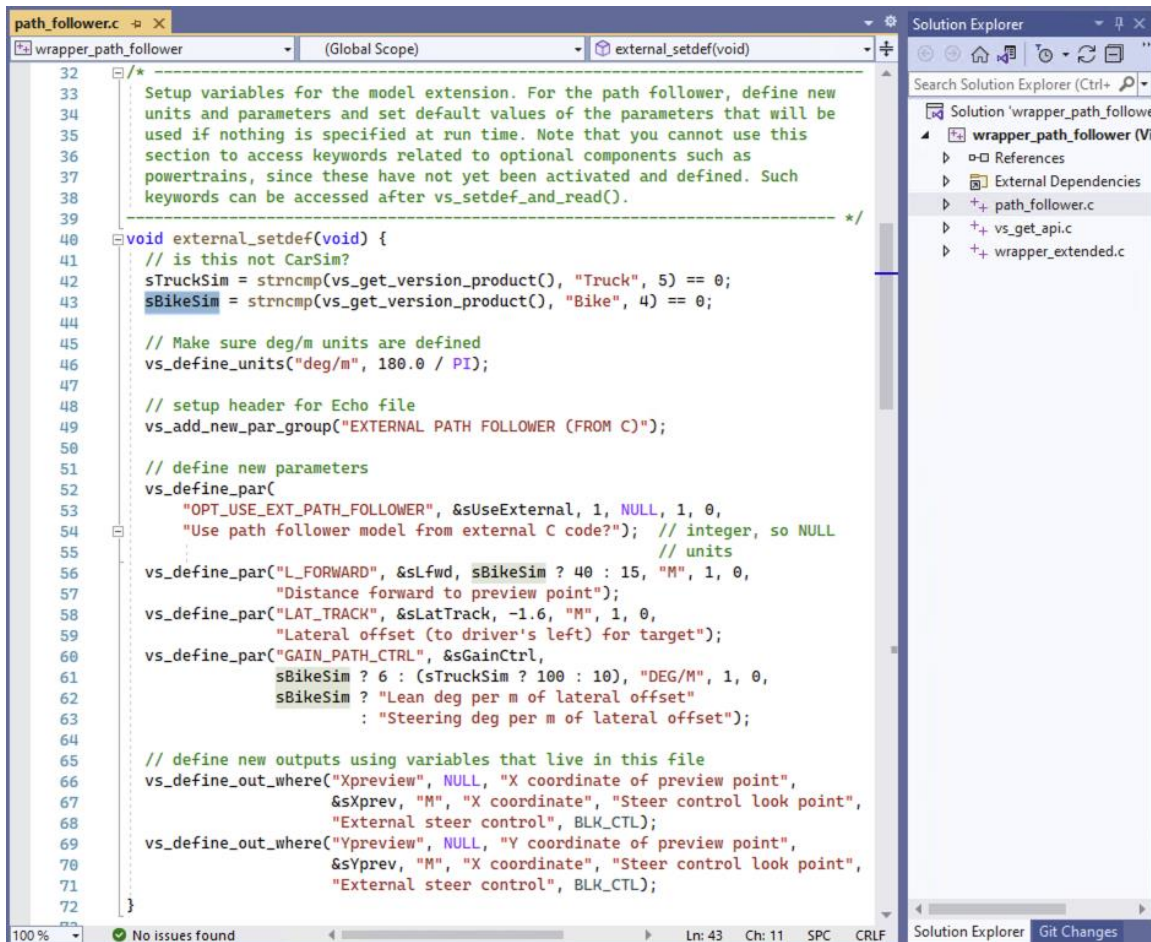


Figure 22. Source code for the function external_setdef.

The function `external_setdef` provides the same information that has been used in other examples:

- Lines 43 and 44 assign Boolean values to the variables `sTruckSim` and `sBikeSim`.
- Line 47 has the API equivalent of the VS Command `DEFINE_UNITS` and installs the “deg/m” units for the steering control gain.
- Lines 55 - 61 define the three parameters used in the VS Command examples: `L_FORWARD`, `LAT_TRACK`, and `GAIN_STEER_CTRL`. Here, the API function `vs_define_par` is used to specify all properties of the parameter that were set before (keyword, default value, keyword for units, description), along with properties that cannot be set with VS Commands (e.g., pointer to an existing variable used for the parameter).
- The variables `sTruckSim` and `sBikeSim` are used to provide alternative values for the parameter `GAIN_PATH_CTRL` using the inline C form `cond ? opt1 : opt2`. For example, in line 59 the default value for `BikeSim` is 40; for `TruckSim` it is 100; otherwise, it is 10.
- Lines 64 - 69 define the two output variables added in the other examples: `Xpreview` and `Ypreview`. Here, the API function `vs_define_out_where` is used. Again, the API function sets all the properties that were set in the VS Commands (keyword, long name, keyword for units, component name), along with a pointer to the variable used for the output, two additional labels (generic name and category name, and a macro `BLK_CTL` that indicates a stage in the timestep where the output will be calculated).

This example adds a few more features not available from VS Commands. A heading is set up for the Echo file (line 50), and the static variable `sUseExternal` is used as an integer parameter to activate or disable the external steering model (lines 53 and 54).

Parameters defined with the API function `vs_define_par` are listed in the Echo file in the order in which they are introduced. Thus, the four parameters defined here (lines 53 – 61) will be shown in the Echo file together, immediately after the header that was installed with the `vs_add_header` function (line 50). For example, Figure 23 shows part of the Echo file generated when using this custom C program.

Performing calculations with external_calc

Figure 24 shows the source code for `external_calc`, the callback function that calculates the control angle and the coordinates of the preview point. This function has two arguments: the current simulation time and an integer macro that identifies the location in the equations. As noted in the comments (and in the *VS API Manual*), this function can be used in many places. In this example, only two are used:

1. `VS_EXT_AFTER_READ` (after the input Parsfiles have been read but before the initialization is performed),
2. `VS_EXT_EQ_IN` (the stage when driver controls are calculated, before the equations for the vehicle multibody model).

```

LastRun_END.PAR #
84 TSTOP          76 ; s ! Stop when this time is reached
85 ! T_DT          0.0005 ; s ! CALC -- Time increment between calculations
86
87 ! -----
88 ! EXTERNAL PATH FOLLOWER (FROM C)
89 ! -----
90 OPT_USE_EXT_PATH_FOLLOWER 1 ! [D] Use path follower model from external C code?
91 L_FORWARD        15 ; m ! [D] Distance forward to preview point
92 LAT_TRACK        -1.6 ; m ! [D] Lateral offset (to driver's left) for target
93 GAIN_PATH_CTRL   10 ; deg/m ! [D] Steering deg per m of lateral offset
94
95 ! -----
96 ! SYSTEM CONSTANTS
97 ! -----
98 ! DR            57.29577951 ; - ! Deg/rad symbol to use in formulas (read-only)
99 ! G             9.80665 ; - ! Symbol for gravity constant m/s/s (read-only)

```

Figure 23. Display of new parameters added to VS model.

```

path_follower.c  wrapper_path_follower  (Global Scope)  external_setdef(void)
72  /*
73  Perform calculations involving the model extensions. This function is called from
74  nine places as defined in vs_deftypes.h.
75  */
76  void external_calc (vs_real t, vs_ext_loc where) {
77  switch (where) {
78  case VS_EXT_AFTER_READ: // after having read the Parsfile inputs
79      sXref = vs_get_var_ptr("X0"); // get pointers to existing model variables
80      sYref = vs_get_var_ptr("Y0");
81      sYaw = vs_get_var_ptr("YAW");
82      sTstart = vs_get_var_ptr("TSTART");
83      if (sBikeSim) {
84          sImpCtrl = vs_get_var_ptr("IMP_GAMMA_TARG"); // target lean angle
85          if (sUseExternal) vs_statement("IMPORT", "IMP_GAMMA_TARG vs_replace", 1);
86      }
87      else {
88          sImpCtrl = vs_get_var_ptr("IMP_STEER_SW"); // steering wheel angle
89          if (sUseExternal) vs_statement("IMPORT", "IMP_STEER_SW vs_replace", 1);
90      }
91
92      break;
93
94  case VS_EXT_EQ_IN: // calculate X and Y, using the state of the vehicle
95      sXprev = *sXref + sLfwd*cos(*sYaw);
96      sYprev = *sYref + sLfwd*sin(*sYaw);
97
98      if (!sUseExternal) ; // no effect if sUseExternal is FALSE
99      else if (t <= *sTstart) *sImpCtrl = 0.0; // no steering at the start
100     else // control angle proportional to the lateral error
101         *sImpCtrl = sGainCtrl*(sLatTrack -vs_road_l(sXprev, sYprev));
102         if (sBikeSim) *sImpCtrl *= -1;
103     break;
104
105     /* unused locations for this example:
106     VS_EXT_EQ_PRE_INIT, VS_EXT_EQ_INIT, VS_EXT_EQ_INIT2, VS_EXT_EQ_STATE,
107     VS_EXT_MIN, VS_EXT_DYN, VS_EXT_EQ_OUT, VS_EXT_EQ_SAVE, VS_EXT_EQ_FULL_STEP,
108     VS_EXT_EQ_END */
109 }
110
111 }

```

Figure 24. Source code for the function external_calc.

Some of the existing variables in the VS Math Model are used, and pointers are obtained for them using the API function `vs_get_var_ptr` after the Parsfile imports have been read, but before any calculations are made with the new path follower (lines 79 – 89). Again, the variable `sBikeSim` is used to include code for getting pointers to variables that differ in BikeSim and the others (e.g., `IMP_GAMMA_TARG` in BikeSim or `IMP_STEER_SW` in the others).

Because an existing Import is used to apply the calculated control angle, it is automatically activated with the `IMPORT VS` Command if the new parameter `sUseExternal` is not zero (lines 85 and 89).

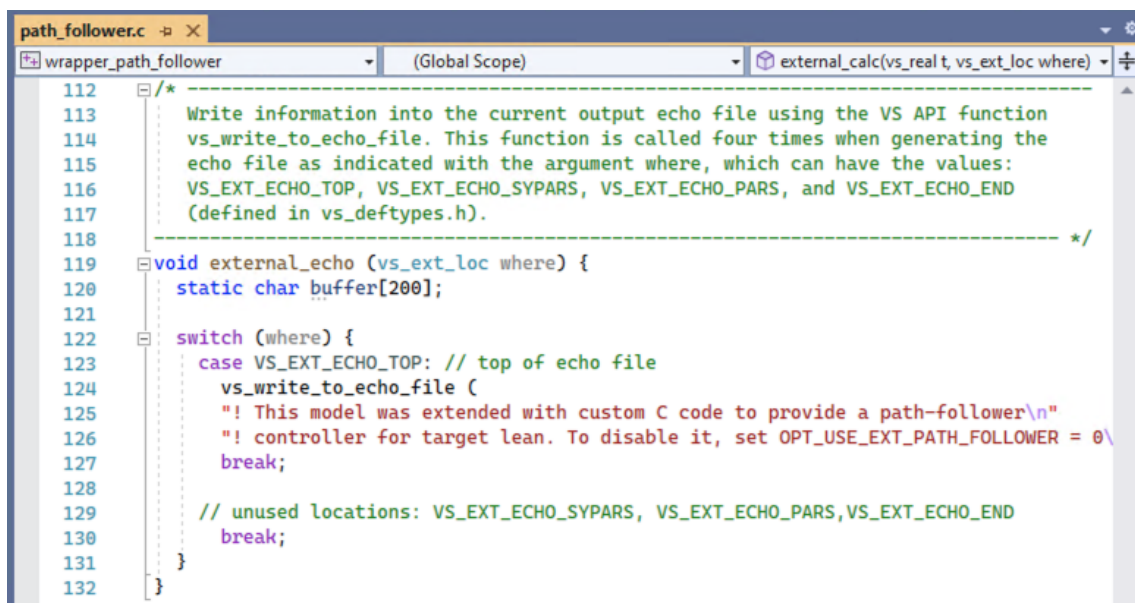
As was the case for the implementation with VS Commands, the two coordinates of the preview point are calculated at the start of each timestep (`VS_EXT_EQ_IN`, lines 95 and 96).

The import control angle is calculated next if the controller is activated (lines 98 - 101). The sign is changed if this is BikeSim (line 102).

Customizing the Echo file

Most extensions to VS Math Models will require only the two functions that are installed with function `vs_install_setdef_function` (Figure 22) and function `vs_install_calc_function` (Figure 24).

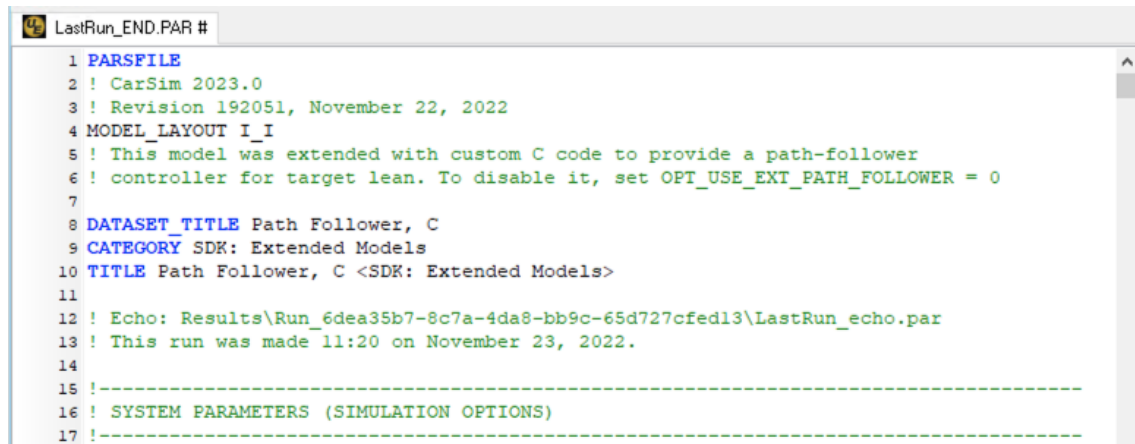
However, the API also supports extensions that might use non-standard input formats or writing into the Echo files. The example shows how the Echo file can be extended to show additional information using custom code, in a function `external_echo` (Figure 25). When the VS Math Model writes an Echo file, it will call this function in four places, identified with the argument `where`. A VS API function `vs_write_to_echo_file` is used to send a string to the DLL so the VS Math Model can in turn write the string to file.



```
path_follower.c
wrapper_path_follower (Global Scope) external_calc(vs_real t, vs_ext_loc where)
112  /*
113   * Write information into the current output echo file using the VS API function
114   * vs_write_to_echo_file. This function is called four times when generating the
115   * echo file as indicated with the argument where, which can have the values:
116   * VS_EXT_ECHO_TOP, VS_EXT_ECHO_SYMPARS, VS_EXT_ECHO_PARS, and VS_EXT_ECHO_END
117   * (defined in vs_deftypes.h).
118   */
119  void external_echo (vs_ext_loc where) {
120      static char buffer[200];
121
122      switch (where) {
123          case VS_EXT_ECHO_TOP: // top of echo file
124              vs_write_to_echo_file (
125                  "! This model was extended with custom C code to provide a path-follower\n"
126                  "! controller for target lean. To disable it, set OPT_USE_EXT_PATH_FOLLOWER = 0\n"
127                  break;
128
129              // unused locations: VS_EXT_ECHO_SYMPARS, VS_EXT_ECHO_PARS, VS_EXT_ECHO_END
130              break;
131      }
132  }
```

Figure 25. Example function `external_echo`.

The code in this example writes additional text at the top of the Echo file to tell users that the run was made with an extended model (lines 124 - 126). This generates information in the Echo file seen in Figure 26 (lines 6 and 7).



```
1 PARSFILE
2 ! CarSim 2023.0
3 ! Revision 192051, November 22, 2022
4 MODEL_LAYOUT I_I
5 ! This model was extended with custom C code to provide a path-follower
6 ! controller for target lean. To disable it, set OPT_USE_EXT_PATH_FOLLOWER = 0
7
8 DATASET_TITLE Path Follower, C
9 CATEGORY SDK: Extended Models
10 TITLE Path Follower, C <SDK: Extended Models>
11
12 ! Echo: Results\Run_6dea35b7-8c7a-4da8-bb9c-65d727cfed13\LastRun_echo.par
13 ! This run was made 11:20 on November 23, 2022.
14
15 ! -----
16 ! SYSTEM PARAMETERS (SIMULATION OPTIONS)
17 ! -----
```

Figure 26. Top of Echo file from example C wrapper program.

As shown earlier, the Echo files generated with the extended model show the new parameters just as if they were built in, with the keyword, value, units, and description (Figure 23, page 27). However, be aware that new output variables are not listed in text files and spreadsheets accessed with the **View** button the **Run Control** screen, or when browsing to select output files for a **Plot Setup** dataset. This is because the VS Browser obtains the documentation files with direct calls to the DLL without extensions from any wrapper programs.

When this wrapper program is run, the same simulation results are obtained as were seen before in the VS Commands example. The only differences in the End Parsfiles involve the display of the new parameters and the notice at the top of the file; all state variables end with the same values in both cases (VS Commands and C).

Summary

This document shows how a VS Math Model can be extended using several methods. The example is a path follower, in which a CarSim or TruckSim vehicle is steered in proportion to the lateral distance between a preview point a fixed distance in front of the vehicle, and the target path (a constant distance to the left or right of a road centerline). In BikeSim, the controller is used to calculate target bike lean.

Overall, five methods were shown, including two methods using MATLAB and two methods using C. The advantages and disadvantages of each method are summarized below.

VS Commands, No External Software

The first approach uses VS Commands and requires no additional software beyond CarSim, TruckSim, or BikeSim. To extend the model with VS Commands:

1. Read through the detailed documentation (readme text files and Echo files) to find the existing Import and output variables of interest. Also, review the Echo file for any existing parameters that would be used in equations you will add to the model.
2. Add variables as needed to the model using VS Commands such as `DEFINE_PARAMETER`, `DEFINE_OUTPUT`, and others.

3. Add equations to the model as needed using VS Commands such as `EQ_IN`, `EQ_OUT`, and others.
4. Add extra information for user-friendliness, such as new units (e.g., “deg/m”) and labels for output variables.

Advantages

1. No additional software is needed. Model extensions may be shared with other users with no restrictions, other than licensing and installation of the VehicleSim product.
2. Commands are tightly integrated into the VS Solver methods. Equations can be inserted in multiple locations in the model equations, allowing the model extensions to interact with the original model during a single timestep.
3. The model extensions are fully documented in the normal Echo files generated for each run. New outputs and import variables are included in machine-generated documents, along with the built-in variables.

Disadvantages

1. The VS Command language is limited to supporting the addition of new variables and equations that can be represented with a single statement. It supports user-defined functions and an IF statement but does not have built-in looping or conditional blocks.
2. If multiple users in a group each add VS Commands for complicated extensions, the management can get more complicated than would be the case if a structured programming language were used.

MATLAB with VS Commands

MATLAB programs can load the VS Solver DLL and apply VS Commands programmatically rather than by using Parsfiles that are read by the VS Math Model. One of the MATLAB examples used the VS API `vs_statement` function to apply the same VS Commands that were used in the example without external software. The results obtained were the same.

Advantage

VS Commands can be applied from within MATLAB and applied for any datasets; it is not necessary to prepare links to Parsfile with the VS Commands.

Disadvantage

It is necessary to have access to MATLAB and to set up M file(s) to apply the VS Commands.

MATLAB with I/O Arrays

External programs can communicate with the VS Math Model each timestep using arrays of Import and Export variables, as is done with Simulink and LabVIEW. This was shown for the example steering controller using MATLAB.

Advantages

1. Details for a model extension are handled in MATLAB, which provides a full programming language.
2. MATLAB model extensions will not complicate the use of VS Commands provided with the conventional method of reading Parsfiles.

Disadvantages

1. It is necessary to have access to MATLAB prepare an M file.
2. The I/O arrays are copied once per timestep, so it is inevitable that any model extension that involves both copying import and export arrays will introduce a 1-step delay in the model behavior.
3. Variables defined in MATLAB will not appear in machine-generated files showing parameters, imports, and outputs, because the VS Browser only has access to the VS Solver library that the variables it generates directly.

C with VS Commands

The C language supports the same method used in MATLAB to apply VS Commands programmatically rather than by using Parsfiles. As with MATLAB, the results obtained from a custom C wrapper program that applied VS Commands were the same.

Advantage

Essential VS Commands will be applied programmatically regardless of how Parsfile datasets are used.

Disadvantage

It is necessary to create a custom C program.

C with VS API Functions

Although API functions may be used with MATLAB, more API options exist when the programming language is C or a language that uses pointers as they are defined in C. VS Solvers are built using ANSI C and rely heavily on VS API functions to support built-in features. The API functions used in a wrapper are often the same ones used internally to create a VS Math Model.

Advantages

1. The external program can be given direct access to internal variables (integer and double precision) in the VS Math Model. Any variable in the model with an associated keyword may be accessed directly in the wrapper program by getting a pointer to the variables with API functions such as `vs_get_var_ptr`.
2. New variables that are physically located in the external program can be installed in the VS Math Model for seamless integration with the built-in capabilities for reading Parsfiles, performing new calculations, and writing Echo files.

3. The integration of a model extension with the built-in model is at least as good as can be done with VS Commands (every location where an `EQ_` VS Command can insert equations is also available as callback location).
4. A wrapper written in C can extend a VS Math Model in many ways that are not possible with VS Commands.

Disadvantages

1. It is necessary to create a custom C program.
2. Variable defined in the custom wrapper will not appear in machine-generated files showing parameters, imports, and outputs, because the VS Browser only has access to the VS Solver library that the variables it generates directly.
3. There is more documentation to read for using the VS API than is the case for VS Commands.